

Overloading di Funzioni

Overloading di Funzioni

Come definire più funzioni con lo stesso nome ma parametri diversi in C++.

Più versioni della stessa funzione

Immagina di voler calcolare l'area di forme diverse: un cerchio, un rettangolo, un triangolo. Potresti chiamare le funzioni `areaDelCerchio`, `areaDelRettangolo`, `areaDelTriangolo`. Ma sarebbe più comodo chiamarle tutte `area` e lasciare al compilatore il compito di capire quale usare.

In C++ questo si chiama **overloading** (sovraccarico): puoi avere più funzioni con lo stesso nome, purché abbiano parametri diversi.

Come funziona

Il compilatore distingue le funzioni guardando il numero e il tipo dei parametri. Quando chiami la funzione, sceglie automaticamente quella giusta:

```

void stampa(int x) {
    cout << "Intero: " << x << endl;
}

void stampa(double x) {
    cout << "Decimale: " << x << endl;
}

void stampa(string s) {
    cout << "Stringa: " << s << endl;
}

int main() {
    stampa(42);           // il compilatore sceglie stampa(int)
    stampa(3.14);         // il compilatore sceglie stampa(double)
    stampa("Ciao");       // il compilatore sceglie stampa(string)
    return 0;
}

```

Output:

```

Intero: 42
Decimale: 3.14
Stringa: Ciao

```

Overloading con numero diverso di parametri

Puoi anche avere versioni con un numero diverso di parametri:

```
int somma(int a, int b) {  
    return a + b;  
}  
  
int somma(int a, int b, int c) {  
    return a + b + c;  
}  
  
int somma(int a, int b, int c, int d) {  
    return a + b + c + d;  
}  
  
int main() {  
    cout << somma(1, 2) << endl;           // 3  
    cout << somma(1, 2, 3) << endl;        // 6  
    cout << somma(1, 2, 3, 4) << endl;    // 10  
    return 0;  
}
```

Esempio pratico: area di figure diverse

Un uso classico è calcolare l'area di figure geometriche diverse con lo stesso nome `area` :

```

#include <iostream>
using namespace std;

const double PI = 3.14159265358979;

// Area del cerchio – riceve solo il raggio
double area(double raggio) {
    return PI * raggio * raggio;
}

// Area del rettangolo – riceve base e altezza
double area(double base, double altezza) {
    return base * altezza;
}

// Area del triangolo – riceve base, altezza e un flag per distinguerla dal
// rettangolo
double area(double base, double altezza, bool triangolo) {
    return (base * altezza) / 2;
}

int main() {
    cout << "Area cerchio (r=5): " << area(5.0) << endl;
    cout << "Area rettangolo (4x6): " << area(4.0, 6.0) << endl;
    cout << "Area triangolo (3x8): " << area(3.0, 8.0, true) << endl;
    return 0;
}

```

Cosa non funziona: il tipo di ritorno non basta

Il compilatore distingue le funzioni solo dai parametri, non dal tipo di ritorno. Questo causa un errore:

```

int calcola(int x) { return x * 2; }
double calcola(int x) { return x * 2.0; } // ERRORE: i parametri sono
identici!

```

Il compilatore non sa quale delle due chiamare, e lo segnala come errore.

Overloading o parametri di default?

A volte si può ottenere lo stesso risultato con entrambi gli approcci:

```
// Con overloading – due funzioni separate
void saluta() { cout << "Ciao, ospite!" << endl; }
void saluta(string nome) { cout << "Ciao, " << nome << "!" << endl; }

// Con parametro di default – una sola funzione
void saluta(string nome = "ospite") {
    cout << "Ciao, " << nome << "!" << endl;
}
```

La regola generale:

- Usa i **parametri di default** quando la funzione fa la stessa cosa con valori diversi
- Usa l'**overloading** quando la funzione fa cose diverse per tipi diversi di input

Esempio pratico: confronto flessibile

```
#include <iostream>
#include <string>
using namespace std;

// Confronta due interi
bool confronta(int a, int b) {
    return a == b;
}

// Confronta due decimali con una piccola tolleranza
// (i numeri decimali raramente sono esattamente uguali)
bool confronta(double a, double b) {
    return (a - b < 0.0001) && (b - a < 0.0001);
}

// Confronta due stringhe
bool confronta(string a, string b) {
    return a == b;
}

int main() {
    cout << boolalpha;
    cout << confronta(5, 5) << endl;           // true
    cout << confronta(3.14, 3.14) << endl;      // true
    cout << confronta("ciao", "ciao") << endl;  // true
    cout << confronta("ciao", "Ciao") << endl;  // false (maiuscole
    contano)
    return 0;
}
```